Assignment 7: Implementing the λ-Calculus

CAS CS 320: Concepts of Programming Languages

due Thursday, October 29 by 8:00PM

For the OCaml problems, you'll put your solutions in the file `lib/assign7.ml`. See the file `test/test_assign7.ml` for example behavior of each function. You'll submit your solutions via Gradescope, where your code will be checked against a collection of hidden tests. You must pass **3/4th** of the Gradescope unit tests on each problem for the assignment to be considered attempted. Your code must also follow the guidelines of the course for it to be considered attempted (i.e., it must use the course standard library, and it may not use any references, arrays, mutable fields, or for/while loops).

For the written problems, you'll submit your solutions via Gradescope as a single PDF file. They must be hand-written and exceptionally neat, and the final answer to each problem must be clear, e.g., surrounded by a box. You must cite your sources per the instructions in the course manual.

You're required to submit solutions to every problem in this assignment to receive full credit, but we will only provide detailed feedback on **bolded** problems. Since the assignment is graded on participation, we ask that you *try at least the bolded problems with as few external aids as possible.* You should think of these problems as simulated quizzes or exams. *You won't be penalized for mistakes*, so it's in your best interest to do these problems on your own; the detailed feedback we provide will generally be more useful (for you and for us) if you submit your own work.

Programming

Outline. This week you'll be building an implementation of the untyped λ-calculus by directly translating a small-step semantics. You'll also be "programming" in the λ-calculus via Church-encodings. This is what you'll need to implement.

```
module Expr : sig
  type t
  val of_sexpr_opt : Sexpr.t -> t option
  val subst : t -> string -> t -> t
  val equal : t -> t -> bool
end
val step : Expr.t -> Expr.t option (* bolded *)
val expr_of_int : int -> Expr.t
val int_of_expr_opt : Expr.t -> int option (* bolded *)
val suc : Expr.t
val add : Expr.t (* bolded *)
```

Syntax

We'll be using S-expressions for the concrete syntax for our λ-calculus interpreter:

```
<expr> ::= ( lambda <var> <expr> )
         | ( <expr> <expr> )
         | <var>
```

where we take a `<var>` to be determined by a nonempty sequence of lowercase English letters.[1] You need to implement the function

```
val of_sexpr_opt : Sexpr.t -> Expr.t option
```

so that `of_sexpr_opt se` is `Some e` if `se` is recognized by the above grammar and represents the expression `e`, and is `None` otherwise.

Typing

There are no typing rules. We're implementing the *untyped* λ-calculus.

[1] Note that this means `lambda` is a valid variable name.

Semantics

We write the following semantic rules using the following abstract syntax for the λ-calculus:

$$e \ ::= \ \lambda x.e \mid ee \mid x$$

You'll be implementing the following small-step semantic rules.

$$\frac{e_1 \text{ is not of the form } \lambda x.e \qquad e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \ \textsc{appLeft}$$

$$\frac{e_1 \text{ is a normal form} \qquad e_2 \longrightarrow e_2'}{e_1 e_2 \longrightarrow e_1 e_2'} \ \textsc{appRight} \qquad \frac{e \longrightarrow e'}{\lambda x.e \longrightarrow \lambda x.e'} \ \textsc{fun}$$

$$\frac{}{(\lambda x.e_1)e_2 \longrightarrow [e_1/x]e_2} \ \textsc{beta}$$

This is a deterministic version of the nondeterministic rules we presented in class, using an evaluation strategy called the *leftmost-outermost* strategy. This strategy has the property that if e has a normal form w.r.t. the nondeterministic rules, then it one w.r.t. this deterministic system as well (this is called the *standardization theorem*). You need to implement the function

```
val step : Expr.t -> Expr.t option
```

so that `step e` is `Some e'` where `e` $\longrightarrow$ `e'` according to the above rules, and `None` if `e` is already in normal form. This will depend on the function

```
val subst : Expr.t -> string -> Expr.t -> Expr.t
```

where `subst e2 x e1` is $[e_2/x]e_1$. Since we're evaluating under binders, *this must be capture-avoiding substitution.* For this, you should use the function `Utils.gensym` to generate fresh variable names.

For testing purposes, it will also be useful to have a function that can compare expressions up to α-equivalence. You need to implement the function

```
val equal : Expr.t -> Expr.t -> bool
```

so that `equal e1 e2` is `true` exactly when $e_1 =_\alpha e_2$.

Church Numerals

The λ-calculus is Turing-complete, i.e., it can be used as a foundations of computation. This is somewhat surprising given we have no data types, just functions. The fundamental observation of Alonzo Church and others is that we can *encode* data types as functions. For example, we represent the natural number n as the

higher-order function which applies a function n times:

$$
\begin{aligned}
0 &\mapsto \lambda f.\lambda x.x \\
1 &\mapsto \lambda f.\lambda x.fx \\
2 &\mapsto \lambda f.\lambda x.f(fx) \\
3 &\mapsto \lambda f.\lambda x.f(f(fx)) \\
&\vdots \\
n &\mapsto \lambda f.\lambda x.f(f(...(fx))\dots) \equiv \lambda f.\lambda x.f^n x
\end{aligned}
$$

We call expressions of the form $\lambda f.\lambda x.f^n x$ *Church numerals* for n. You need to implement the functions

```
val expr_of_int : int -> Expr.t
val int_of_expr_opt : Expr.t -> int option
```

where `expr_of_int n` is the Church numeral for n, and `int_of_expr_opt e` is `Some n` if `e` is α-equivalent to the Church numeral for n (and `None` otherwise).

Finally, an encoding is no good if we can define standard functions on the underlying data. Your last task is to "program" in the λ-calculus. You need to construct an expression `suc` which is a λ-calculus expression for adding 1 to a Church numberal, and an expression `add` which is a λ-expression for adding two Church numerals. So, for example, the following assertions should hold.

```
let two = expr_of_int 2
let prog1 = Expr.app (suc, two)
let three = multistep prog
let _ = assert (int_of_expr_opt three = Some 3)
let prog2 = Expr.app (Expr.app (add, two), three)
let five = multistep prog
let _ = assert (int_of_expr_opt five = Some 5)
```

Note that you can use `Expr.of_string` instead of building the ADT:

```
let example = Expr.of_string "(lambda f (lambda x (f x)))"
```

Written Problems

You're required to submit solutions to the following problems from the lecture notes on the λ-calculus (from October 22): 1, **2**, 4, 5, 6, **9**, 10, 13, **15**